



Security Assessment Final Report



ReCLAMM 2.1 Review

April 2026

Prepared for Balancer Labs

Table of content

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Threat model	5
Scope & assets	5
Primary actors	5
Risks and threats	7
1. Price Manipulation Through Trading	7
2. Adversarial Governance	7
3. Pool Bricking via Near-Total Withdrawal	8
4. Malicious Hook Registration	9
5. Initialization Replay	9
6. BPT Rate Oracle Exploitation	9
7. Arithmetic Safety and Rounding	9
Analysis of pool calculations and safeguards	10
1. Invariant calculation	10
2. Swap Calculations	10
3. Virtual Balance Updates	10
3a. Per-block Freeze	11
3b. Price Ratio Interpolation (computeVirtualBalancesUpdatingPriceRatio)	11
3c. Price Range Shift (computeVirtualBalancesUpdatingPriceRange)	11
4. Liquidity Proportional Scaling	12
5. Price Ratio Governance Safeguards	13
6. Centeredness Margin Safeguard	13
7. Summary of Rounding Policy	13
Findings Summary	15
Severity Matrix	15
Detailed Findings	16
Informational Severity Issues	17
I-01: Swap Fee and shift rate relationship is documented but not enforced in code	17
I-02: _ensureVaultIsInitialized misleadingly named, it checks pool initialization, not vault	19
I-03: Paused pools cannot be safely unpaused due to stale virtual balances	20
Disclaimer	21
About Certora	21

Project Summary

Project Scope

Project Name	Repository (link)	Commit Hash	Platform
ReCLAMM	reclamm	f7f5f5c (initial) cff1803 (latest)	EVM

Project Overview

This document describes the verification of **ReCLAMM 2.1** using manual code review. The work was undertaken from **9th of April 2026** to **23rd of April 2026**.

The following contract list is included in our scope:

```
contracts/ReClammPool.sol
contracts/ReClammPoolFactory.sol
contracts/ReClammPoolHelper.sol
contracts/lib/ReClammMath.sol
contracts/lib/ReClammPoolFactoryLib.sol
contracts/interfaces/IReClammPool.sol
```

During the audit and fix review period Balancer made several pull requests with code optimisations and small code refactoring. We have reviewed those up to commit [1726262](#) and have found no issues arising from those changes.

Protocol Overview

The ReCLAMM is a liquidity pool design based on the constant product model, enhanced with virtual balances to concentrate liquidity within a dynamic price range. Unlike traditional concentrated liquidity pools, where liquidity providers must actively manage their positions as prices move, ReCLAMM automatically readjusts the active price range. This enables passive liquidity provision while maintaining capital efficiency and targeted exposure near the market price. The pool is configured through three parameters: the price range ratio, which defines the width of the concentrated liquidity bracket; the daily price shift rate, which controls how quickly

the bracket moves with market price changes; and the centeredness margin, which defines the target range — the pool is in range when its centeredness is at or above the margin. When the centeredness falls below the margin, the pool is outside the target range, and the price range starts to move toward the current market price.

The new ReCLAMM pool version 2.1 includes a restructuring of the code and adds extra safeguards.

Changes include:

1. **Added hook call validation**

Hook flags, `onRegister`, and all hook callbacks (`onBeforeRemoveLiquidity`, `onBeforeAddLiquidity`, `onBeforeInitialize`) are implemented entirely within the pool itself. `onRegister` is a view function that explicitly verifies `pool == address(this)`, preventing a malicious contract from registering these hooks for itself.

2. **Separate initialization functions from the main pool**

The new **ReClammPoolHelper** contract is a stateless companion contract that handles computations used at initialization and off-chain: calculating initial balance ratios, validating initialization prices, and computing the current spot price. This keeps the main pool's bytecode smaller and its logic focused on the critical swap path.

3. **Minimum Price Ratio Guard on `startPriceRatioUpdate`**

`startPriceRatioUpdate` enforces a minimum price ratio (`MIN_PRICE_RATIO = 1.0001e18`), preventing governance from setting an end price ratio that would collapse the price range to near-zero.

4. **Rounding Consistency Fix in `ReClammMath`**

In `computePriceRatio`, the calculations are refactored to prevent overflows, and final division rounds down, consistent with `computeTheoreticalPriceRatioAndBalances`, ensuring price ratio calculations are consistent between initialization and ongoing operation.

Threat model

In the following section, we define the threats and risks associated with the ReClamm pool and other parts of the Balancer protocol. For each we have determined how Balancer has mitigated these risks.

Scope & assets

Protocol components in scope

- Pool creation using the `ReClammPoolFactory`
- ReClamm pool implementation
- Interaction between the pool and Balancer V3 vault, including add/remove liquidity operations, swaps and hooks.

High-value assets

- Token balances held in Vault (user LP funds).
- Protocol-owned tokens (protocol Fees)
- BPT tokens held by users

Primary actors

Pool Creator

The pool creator deploys the pool through `ReClammPoolFactory` and supplies all immutable initialization parameters: the initial min/max/target prices, the daily price shift exponent, the centeredness margin, and the token rate flags. These are baked into the contract as immutables and cannot be changed after deployment. The creator also triggers pool initialization through the vault, which derives and commits the initial virtual balances and price ratio state via `onBeforeInitialize`.

The creator has no privileged role after initialization. Their risk surface is entirely front-of-lifecycle: misconfigured initial prices or inconsistent rate flags produce a pool that is permanently mis-priced from block one.

Swap Fee Manager / Governance

When a swap fee manager is designated, they hold ongoing authority over the pool's dynamic parameters. When none is set, governance inherits this role. Together they control:

- `setCenterednessMargin`: redefines the threshold above which price shift is suppressed.
- `setDailyPriceShiftExponent`: controls the speed at which the price range slides when out of range.
- `startPriceRatioUpdate` / `stopPriceRatioUpdate`: schedules or cancels a gradual change to the price ratio, subject to minimum duration and maximum daily rate constraints.

Honest operation of this role is a stated trust assumption of the system.

The Vault

The Balancer V3 Vault is the sole caller of `onSwap`, `onBeforeInitialize`, `onBeforeAddLiquidity`, and `onBeforeRemoveLiquidity`, all of which are gated by `onlyVault`. Live balances received from the Vault are used directly in virtual balance and swap calculations without further validation — a compromised Vault could corrupt pool state by passing arbitrary values.

Vault Users

Liquidity providers add and remove liquidity proportionally. Unbalanced operations are disabled at registration time. **Traders** submit swaps through a router. These include normal users, MEV/arbitration bots and interacting protocols.

Risks and threats

1. Price Manipulation Through Trading

Attackers should not be able to exploit flash loans, intra-block interactions, or the pool's price shift mechanism to extract value at the expense of liquidity providers or other users.

The balances passed to onSwap come from the Vault's own internal accounting and cannot be inflated by the caller. Virtual balance updates are frozen within a block — once the first interaction in a block updates the timestamp, all subsequent calls in the same block see the same virtual balances — which limits the state an attacker can create and exploit within a single transaction. Swap fees further ensure that round-trip repricing extraction is unprofitable at the configured shift rates.

The pool's out-of-range price shift mechanism creates a per-block price movement that could theoretically be extracted by buying the undervalued token before the shift and selling back after. This is kept unprofitable by calibrating the minimum swap fee against the configured shift rate: at the minimum fee of 0.001%, daily shift rates up to 5% remain safe. Operators who increase the shift rate are required to verify that the swap fee satisfies the corresponding minimum — a relationship documented explicitly in the interface NatSpec, but not enforced in the code, as noted in [I-01](#).

2. Adversarial Governance

A compromised or malicious governance key (or swap fee manager) should not be able to manipulate pool prices, trap liquidity, or extract value by abusing the pool's configuration functions.

The most sensitive levers available to governance are the centeredness margin, the price ratio, and the daily price shift exponent. For each, the implementation enforces limits that bound the impact even if the key is fully compromised.

Setting the centeredness margin to an excessively high value would immediately push the pool out of range, triggering an aggressive price shift and allowing the attacker to profit from the resulting movement. This is prevented by requiring that the new margin does not exceed the current live centeredness at the time of the call. A block-level coordinated attack — swap to

improve centeredness, set margin, swap back in the same block — could bypass this check, since virtual balances are frozen after the first interaction.

A malicious or compromised governance actor could use this to raise the centeredness margin above the pool's current centeredness, immediately classifying the pool as out-of-range and triggering the price shift mechanism. However, the `MAX_DAILY_PRICE_SHIFT_EXPONENT` cap bounds the speed at which virtual balances can move, and the minimum swap fee is explicitly sized to ensure that the cost of a round-trip swap exceeds the price improvement available within any single block.

Initiating a price ratio update creates a transition window during which arbitrageurs can skew pool centeredness before the new ratio settles, locking in that skewed state. The extractable value is bounded by three independently enforced limits: the absolute ratio bounds (`MIN_PRICE_RATIO`, `MAX_PRICE_RATIO`), the minimum update duration of one day, and a maximum daily update rate computed with conservative round-up arithmetic. At the maximum allowed daily widening, a swap fee of approximately 2% is required to fully neutralise the extraction.

Increasing the daily price shift exponent raises the pool's exposure to round-trip repricing extraction as discussed in the previous section. This is limited by a factory-validated maximum exponent and by the requirement that the swap fee is sufficient to cover the resulting shift rate. The minimum fee bound is immutable and enforced at the Vault level.

All governance functions require the pool to be initialised and some functions also require the Vault to be locked, ensuring they cannot be called in unusual states. The fee limits and absolute bounds are set at construction time and are themselves immutable.

3. Pool Bricking via Near-Total Withdrawal

A sole LP burning nearly all BPT in a single proportional withdrawal can cause integer truncation of one or both virtual balances to zero, making the pool permanently unusable since subsequent price computations would divide by zero. The `onBeforeRemoveLiquidity` hook explicitly checks that neither post-scaling virtual balance is zero and reverts with `ZeroVirtualBalance` if either would be. The Vault's `POOL_MINIMUM_TOTAL_SUPPLY` floor provides an additional lower bound on the minimum virtual balance after any withdrawal.

4. Malicious Hook Registration

A malicious contract could attempt to register the ReClamm hook contract as its own hook implementation, gaining access to the state-mutating hook functions under a different pool address. `onRegister` prevents this by returning true only when `pool == address(this)`, ensuring the hook can never be shared across or reused by an external pool.

5. Initialization Replay

Calling `onBeforeInitialize` a second time on an already-operational pool would overwrite virtual balances and the price ratio state, potentially resetting the pool to an arbitrary configuration. This is unconditionally blocked by checking `_vault.isPoolInitialized(address(this))` at function entry — a check that cannot be bypassed by any role.

6. BPT Rate Oracle Exploitation

Integrating protocols that call `getRate()` expecting a meaningful BPT exchange rate could make financial decisions based on a value that is manipulable via balance changes. Because the ReClamm invariant shifts continuously with the price range and ratio updates, a BPT rate would be inherently unreliable. `getRate()` therefore unconditionally reverts with `ReClammPoolBptRateUnsupported`, enforcing this constraint at the contract level.

7. Arithmetic Safety and Rounding

Systematic rounding bias or arithmetic edge cases should not allow value to leak from the Vault or destabilise the pool's invariant. All arithmetic uses audited fixed-point libraries with rounding directions chosen consistently to favour the Vault. Duration inputs are capped to prevent exponentiation overflow, and denominator terms are constructed with round-up arithmetic to guarantee positivity at boundary conditions. A detailed treatment is provided in the [Analysis of Pool Calculations and Safeguards](#) section.

Analysis of pool calculations and safeguards

1. Invariant calculation

The ReClamm pool uses a constant-product invariant augmented with virtual balances:

$$L = (R_a + V_a)(R_b + V_b)$$

Unlike standard constant-product AMMs, no square root is taken. This simplifies the virtual balance update formulas but means the invariant is not a linear measure of liquidity and is not suitable for BPT rate computation (hence `getRate()` always reverts). The rounding direction of the invariant is parameterised so it uses the correct rounding direction on each call.

2. Swap Calculations

ExactIn (`computeOutGivenIn`)

$$A_o = \frac{(B_o + V_o) \cdot A_i}{B_i + V_i + A_i}$$

The output is computed with plain integer division, which truncates toward zero. This rounds `amountOut` **down**, meaning the user receives slightly less than the theoretical amount — directly favouring the Vault. A hard check `amountOut <= balancesScaled18[tokenOut]` prevents the pool from being drained below zero real balance.

ExactOut (`computeInGivenOut`)

$$A_i = \frac{(B_i + V_i) \cdot A_o}{B_o + V_o - A_o}$$

The input is computed with `mulDivUp`, rounding the result **up**. The user pays at least the theoretical amount — directly favouring the Vault. The same drain-prevention check is applied before the arithmetic to avoid a subtraction underflow in the denominator.

3. Virtual Balance Updates

Virtual balances are recalculated on every pool interaction via `computeCurrentVirtualBalances`. The function applies three stages in order.

3a. Per-block Freeze

If `lastTimestamp == block.timestamp`, the function returns the stored virtual balances unchanged. This prevents within-block manipulation: a single block can only produce one VB update regardless of how many transactions it contains. The first interaction of each block captures the accumulated time-based drift; all subsequent ones in the same block operate on that frozen snapshot.

3b. Price Ratio Interpolation (`computeVirtualBalancesUpdatingPriceRatio`)

When an active price ratio update is in progress (`currentTimestamp > startTime` and `lastTimestamp < endTime`), the current fourth root of the price ratio is interpolated geometrically:

$$P_c = P_s \cdot \left(\frac{P_e}{P_s} \right)^{\frac{T_c - T_s}{T_e - T_s}}$$

This geometric interpolation (operating on the fourth root, not the ratio itself) produces uniform multiplicative steps over time, analogous to Balancer's gradual weight change mechanism. The virtual balances are then recalculated using a closed-form Bhaskara (quadratic) solution that satisfies the simultaneous constraints of the target price ratio and the preserved current centeredness:

$$V_u = \frac{R_u \left(1 + C + \sqrt{1 + C(C + 4Q_0 - 2)} \right)}{2(Q_0 - 1)}$$

where $Q_0 = \sqrt{\text{priceRatio}}$, C is centeredness, R_u is the real balance of the undervalued token, and

V_u is the resulting virtual balance for that token. The overvalued virtual balance is derived by

preserving the stored ratio

$$V_{\text{overvalued}} / V_{\text{undervalued}}$$

Boundary behaviour: `computeFourthRootPriceRatio` returns the end value for `currentTime >= endTime` and the start value for `currentTime <= startTime`.

A minimum clamp $\max(\min(P_s, P_e), P_c)$ is applied after interpolation to guard against arithmetic undershoot due to `powDown` rounding.

3c. Price Range Shift (`computeVirtualBalancesUpdatingPriceRange`)

When the pool is outside the target range (centeredness < margin), the price range is shifted toward the market by decaying the overvalued virtual balance:

$$V'_o = V_o \cdot (\text{dailyPriceShiftBase})^{T_c - T_l}$$

The duration is capped at 30 days to prevent `powDown` overflow after an extended period without interactions. The undervalued virtual balance is then recomputed from the invariant geometry to maintain the price ratio:

$$V_u = \frac{R_u(V_o' + R_o)}{(Q_o - 1) \cdot V_o' - R_o}$$

Minimum clamp on V_o' : Before computing the denominator, V_o' is clamped from below to:

$$\frac{R_o}{\sqrt[4]{Q_o} - 1}$$

which corresponds to centeredness = 1 (fully centered). `divUp` and `mulUp` are used for the denominator term to ensure it remains strictly positive even at boundary values, preventing a division by zero.

4. Liquidity Proportional Scaling

On proportional add and remove, virtual balances are scaled by the same BPT ratio as real balances:

$$V' = V \cdot \frac{\text{newSupply}}{\text{oldSupply}}$$

Integer division is used (rounds down), which is intentional: slightly lower virtual balances after an add means the pool is marginally more conservative in subsequent swap pricing, favouring the Vault.

A dedicated zero-check after the remove scaling reverts the transaction if either virtual balance would round to zero. Without this check, a near-total withdrawal could truncate a virtual balance to zero, causing a division-by-zero in `computePriceRatio` on the next interaction and permanently bricking the pool.

5. Price Ratio Governance Safeguards

The `startPriceRatioUpdate` function enforces three independent limits before committing a new price ratio target:

Safeguard	Mechanism
Absolute bounds	Reverts if <code>endPriceRatio</code> is outside <code>[MIN_PRICE_RATIO, MAX_PRICE_RATIO]</code>
Minimum delta	Reverts if <code>(priceRatioDelta < 1e6)</code> , preventing degenerate near-zero updates
Maximum rate	Reverts if the effective daily change factor computed as $(\text{max/min})^{86400/\text{duration}}$ using <code>divUp</code> and <code>powUp</code> to round the rate up, exceeds <code>_MAX_DAILY_PRICE_RATIO_UPDATE_RATE</code>

The rate calculation uses round-up arithmetic throughout so that borderline updates are more likely to be rejected rather than incorrectly permitted.

6. Centeredness Margin Safeguard

`setCenterednessMargin` requires that the new margin does not exceed the current live centeredness. This prevents a governance actor from setting a margin so high that the pool is immediately classified as out-of-range, which would force an aggressive price shift and potentially be used to manipulate the spot price. The check is acknowledged as best-effort (it can be bypassed by a settled swap immediately before and after), so it protects against misconfigurations rather than adversarial governance.

7. Summary of Rounding Policy

Operation	Direction	Rationale
<code>amountOut (ExactIn)</code>	Round down (integer division)	User receives less
<code>amountIn (ExactOut)</code>	Round up (<code>mulDivUp</code>)	User pays more

Virtual balances on add/remove	Round down	Conservative VBs improve swap pricing for Vault
Daily rate check	Round up (divUp , powUp)	Borderline—fast updates are rejected
denominator in price range shift	Round up (mulUp)	Ensures denominator remains positive
Fourth root interpolation	Round down (powDown)	Minimum clamp prevents undershoot

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	3	3	1
Total	3	3	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
I-01	Swap Fee and shift rate relationship is documented but not enforced in code	Informational	Acknowledged
I-02	_ensureVaultIsInitialized misleadingly named, it checks pool initialization, not vault	Informational	Fixed in PR198 .
I-03	Paused pool cannot be safely unpaused due to stale virtual balances	Informational	Acknowledged

Informational Severity Issues

I-01: Swap Fee and shift rate relationship is documented but not enforced in code

Files:

contracts/ReClammPool.sol

Description:

The project documentation specifies a safe breakeven relationship between the daily price shift exponent and the swap fee percentage to prevent arbitrage. However, this safety rule is only written in the comments and is not enforced in the smart contract code. The function `setDailyPriceShiftExponent`, which sets the daily price shift exponent, and the function `setStaticSwapFeePercentage` via `VaultAdmin`, which sets the swap fee, are independent. They do not check each other. Because of this, an operator could accidentally or maliciously set a very low fee with a very high shift rate. If this happens, the breakeven time could drop below the network block time, allowing MEV searchers to extract value continuously.

The same defense in depth reasoning also applies to `startPriceRatioUpdate`. A 2x widening at the maximum allowed daily rate requires a swap fee of at least ~2% to neutralize round-trip MEV extraction, and smaller widenings require proportionally lower fees. The current implementation only enforces a hardcoded `_MAX_DAILY_PRICE_RATIO_UPDATE_RATE` and a minimum `updateDuration`, but it does not cross-validate the effective update rate against the current `staticSwapFeePercentage` returned by the Vault. An operator could therefore start a price ratio update at the maximum permitted rate while the pool is configured with the minimum swap fee (0.001%), leaving the pool exposed to profitable MEV round-trips for the entire update window.

Although the project expects that the swap fee manager will set these values correctly, as a defense-in-depth measure, it is safer to enforce this rule in the code. This protects the protocol in case the swap fee manager is compromised or makes a mistake.

Recommendations:

Consider adding a requirement in the `setDailyPriceShiftExponent` function to cross-validate the current swap fee.

Consider adding code in `getMinimumSwapFeePercentage` (which is called during `setStaticSwapFeePercentage`) to also cross-validate the daily price shift. The contract could calculate the breakeven time and revert if it is shorter than a safe threshold, e.g. 60 seconds.

Consider adding a requirement in the `startPriceRatioUpdate` function to cross-validate the effective daily price ratio update rate against the pool's current `staticSwapFeePercentage`. The contract should revert if the configured swap fee is lower than the minimum safe fee derived from the requested update rate, applying the same proportional relationship described in the NatSpec. As a symmetrical guard, `getMinimumSwapFeePercentage` could also take any active price ratio update into account when lowering the swap fee.

Customer Response:

Acknowledged. The Balancer team decided to use a lower max price shift exponent (that is 50%) than the current one (100%). Balancer implemented the change in [pull request #199](#)

Fix Review:

We reviewed and confirmed that pull request #199 looks good. Reducing `MAX_DAILY_PRICE_SHIFT_EXPONENT` from `100e16` to `50e16` is a conservative tightening that lowers the maximum daily price-shift rate from $2x$ to $\sqrt{2}x$ per day and introduces no new issues.

I-02: `_ensureVaultIsInitialized` misleadingly named, it checks pool initialization, not vault

Files:

contracts/ReClammPool.sol

Description:

`_ensureVaultIsInitialized` checks if the pool is initialized at the vault, but the name implies that it checks if the vault is initialized.

Recommendations:

Rename `_ensureVaultIsInitialized` to `_ensurePoolsInitializedAtVault` to accurately reflect what is being checked.

Customer Response:

Fixed in [pull request 198](#).

The function is renamed to `_ensurePoolsInitialized`

I-03: Paused pools cannot be safely unpaused due to stale virtual balances

Files:

contracts/ReClammPool.sol

Description:

When a pool is paused, liquidity can still be removed after recovery mode is enabled by anyone, but this path does not trigger the hooks responsible for updating virtual balances. As a result, the pool's virtual balances may diverge from its actual state while paused, causing the implied price to become stale.

If the pool is later unpaused, operations may resume against out-of-sync virtual balances, which could lead to incorrect pricing behavior. This issue is informational in practice, as pausing is generally reserved for emergency scenarios, including compromised tokens or a compromised pool, and such pools are typically not expected to be unpaused.

Customer Response:

Acknowledged. [Pull request 192](#) adds extra documentation to clarify that Recovery mode is really not recoverable for ReClamm pools and should be used as a one-way operation.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the rarest & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.